

EXPERIMENT - 23

1. Problem Statement

Write a C program that displays a menu and uses a switch statement to print the basic square or rectangular pattern selected by the user. The patterns are to be built with nested loops only, without using arrays, and each pattern is to be written in a separate parameterless function that returns no value. The four patterns are:

- **Solid square pattern** — read the number of rows n and print a solid square of stars having n rows and n columns.
- **Solid rectangular pattern** — read the number of rows m and the number of columns n and print a solid rectangle of stars.
- **Row-number pattern** — read m and n and print a rectangular number pattern in which every element of a row is equal to its row number.
- **Column-number pattern** — read m and n and print a rectangular number pattern in which every element of a column is equal to its column number.

The menu is to be shown again and again until the user chooses to exit, and an error message is to be displayed for an invalid menu choice.

2. Algorithm

Main Program

Step 1: START

Step 2: Display the menu: 1. Solid square pattern, 2. Solid rectangular pattern, 3. Row-number pattern, 4. Column-number pattern, 5. Exit.

Step 3: Read choice.

Step 4: Using a switch statement on choice:

 If choice is 1, call the function `solidSquare()`.

 If choice is 2, call the function `solidRectangle()`.

 If choice is 3, call the function `rowNumberPattern()`.

 If choice is 4, call the function `columnNumberPattern()`.

 If choice is 5, display "Exiting the program."

 For any other value, display "Invalid choice! Enter 1 to 5."

Step 5: Repeat from Step 2 while choice is not equal to 5.

Step 6: STOP

Function `solidSquare()`

Step 1: Read n . While n is less than or equal to 0, display an error message and read n again.

Step 2: Repeat for $i = 1$ to n :

 Repeat for $j = 1$ to n : print a star.

 Move to a new line.

Step 3: Return to the calling function.

Function solidRectangle()

Step 1: Read m. While m is less than or equal to 0, display an error message and read m again.

Step 2: Read n. While n is less than or equal to 0, display an error message and read n again.

Step 3: Repeat for i = 1 to m:

Repeat for j = 1 to n: print a star.

Move to a new line.

Step 4: Return to the calling function.

Function rowNumberPattern()

Step 1: Read m and n, validating each one as above.

Step 2: Repeat for i = 1 to m:

Repeat for j = 1 to n: print the value of i.

Move to a new line.

Step 3: Return to the calling function.

Function columnNumberPattern()

Step 1: Read m and n, validating each one as above.

Step 2: Repeat for i = 1 to m:

Repeat for j = 1 to n: print the value of j.

Move to a new line.

Step 3: Return to the calling function.

3. Test Case Table

#	Choice	Input	Expected Output	Actual Output	Result
1	1 — Square	n = 4	4 rows, each having 4 stars	As expected	PASS
2	2 — Rectangle	m = 3, n = 5	3 rows, each having 5 stars	As expected	PASS
3	3 — Row number	m = 4, n = 4	1 1 1 1 / 2 2 2 2 / 3 3 3 3 / 4 4 4 4	As expected	PASS
4	4 — Column number	m = 4, n = 4	1 2 3 4 repeated in all 4 rows	As expected	PASS
5	2 — Rectangle	m = 5, n = 3	5 rows, each having 3 stars (taller than wide)	As expected	PASS
6	3 — Row number	m = 2, n = 6	1 1 1 1 1 1 / 2 2 2 2 2 2	As expected	PASS
7	1 — Square	n = 1	A single star (smallest square)	As expected	PASS
8	3 — Row number	m = 1, n = 1	A single 1	As expected	PASS
9	1 — Square	n = -3, then 0, then 5	Error message twice, then a 5 × 5 square	As expected	PASS

#	Choice	Input	Expected Output	Actual Output	Result
10	2 — Rectangle	m = -3 then 2, n = -1 then 3	Error message for rows, error message for columns, then a 2 × 3 rectangle	As expected	PASS
11	9 — Invalid	choice = 9	Invalid choice! Enter 1 to 5.	As expected	PASS
12	5 — Exit	choice = 5	Exiting the program, and the loop ends	As expected	PASS

4. Source Code

patterns.c

```
/* Menu-driven program that uses a switch statement to print the basic
   square and rectangular patterns chosen by the user. Every pattern is
   written in its own parameterless function that returns nothing, and
   each one is built with nested loops only. No arrays are used. */

#include <stdio.h>

void solidSquare(void);
void solidRectangle(void);
void rowNumberPattern(void);
void columnNumberPattern(void);

int main(void)
{
    int choice;

    do {
        printf("\n1. Solid square pattern\n");
        printf("2. Solid rectangular pattern\n");
        printf("3. Row-number pattern\n");
        printf("4. Column-number pattern\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: solidSquare(); break;
            case 2: solidRectangle(); break;
            case 3: rowNumberPattern(); break;
            case 4: columnNumberPattern(); break;
            case 5: printf("Exiting the program.\n"); break;
            default: printf("Invalid choice! Enter 1 to 5.\n"); break;
        }
    } while (choice != 5);

    return 0;
}

/* Pattern 1: n rows and n columns of stars. */
void solidSquare(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);
    while (n <= 0) {
        printf("Error: rows must be a positive integer. Enter again: ");
        scanf("%d", &n);
    }

    printf("Solid Square Pattern (n = %d)\n", n);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++)
            printf("* ");
        printf("\n");
    }
}

/* Pattern 2: m rows and n columns of stars. */
void solidRectangle(void)
{
    int m, n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &m);
    while (m <= 0) {
        printf("Error: rows must be a positive integer. Enter again: ");
```

```

        scanf("%d", &m);
    }

    printf("Enter the number of columns: ");
    scanf("%d", &n);
    while (n <= 0) {
        printf("Error: columns must be a positive integer. Enter again: ");
        scanf("%d", &n);
    }

    printf("Solid Rectangular Pattern (m = %d, n = %d)\n", m, n);
    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++)
            printf("* ");
        printf("\n");
    }
}

/* Pattern 3: every element of a row is the row number itself. */
void rowNumberPattern(void)
{
    int m, n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &m);
    while (m <= 0) {
        printf("Error: rows must be a positive integer. Enter again: ");
        scanf("%d", &m);
    }

    printf("Enter the number of columns: ");
    scanf("%d", &n);
    while (n <= 0) {
        printf("Error: columns must be a positive integer. Enter again: ");
        scanf("%d", &n);
    }

    printf("Row-Number Pattern (m = %d, n = %d)\n", m, n);
    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++)
            printf("%d ", i);
        printf("\n");
    }
}

/* Pattern 4: every element of a column is the column number itself. */
void columnNumberPattern(void)
{
    int m, n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &m);
    while (m <= 0) {
        printf("Error: rows must be a positive integer. Enter again: ");
        scanf("%d", &m);
    }

    printf("Enter the number of columns: ");
    scanf("%d", &n);
    while (n <= 0) {
        printf("Error: columns must be a positive integer. Enter again: ");
        scanf("%d", &n);
    }

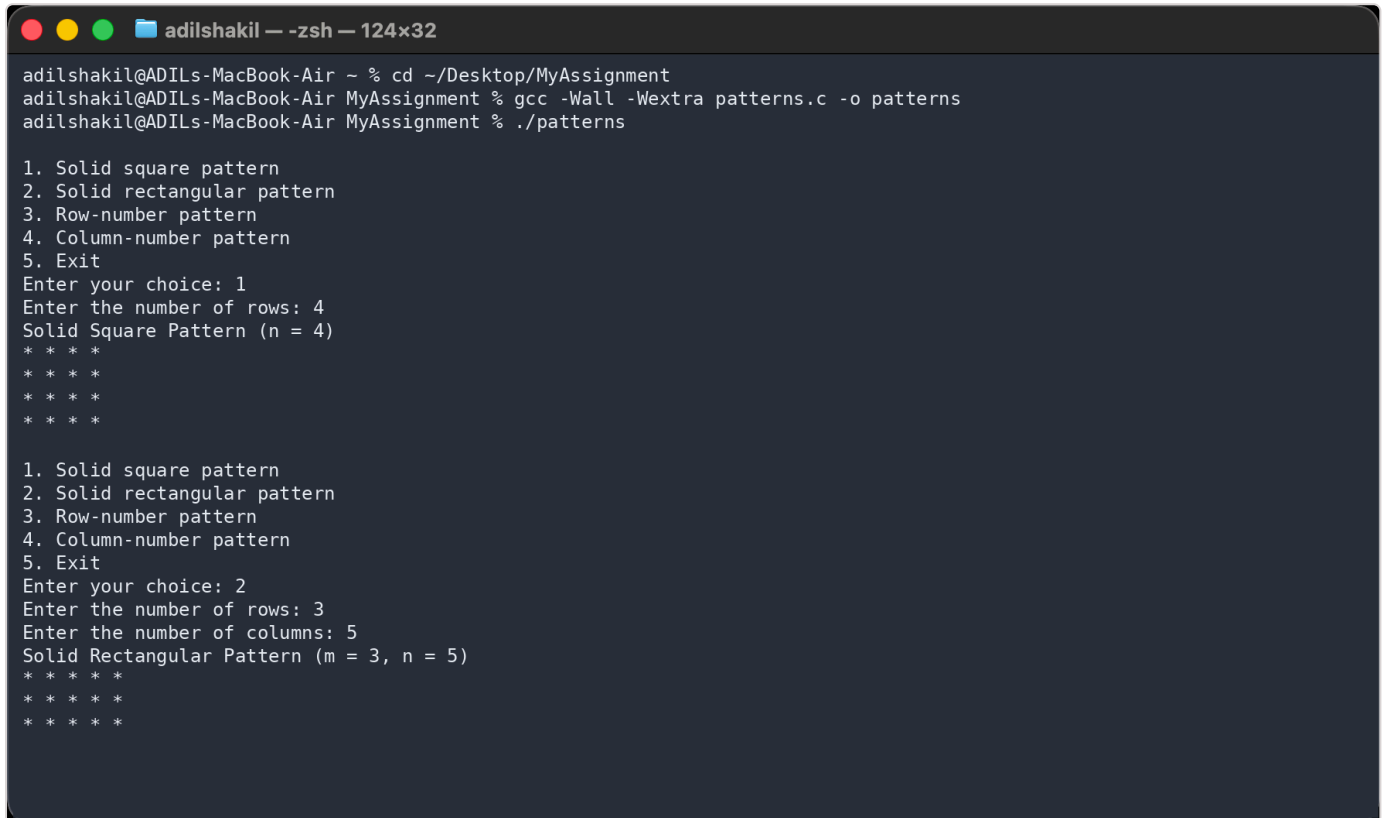
    printf("Column-Number Pattern (m = %d, n = %d)\n", m, n);
    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++)
            printf("%d ", j);
        printf("\n");
    }
}

```

5. Compilation and Output

The program was compiled on the terminal with the warning flags turned on, using the command `gcc -Wall -Wextra patterns.c -o patterns`, and it compiled without a single warning. The screenshots below are of the actual runs on my machine.

Patterns 1 and 2 (stars)



```
adilshakil — zsh — 124x32
adilshakil@ADILs-MacBook-Air ~ % cd ~/Desktop/MyAssignment
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra patterns.c -o patterns
adilshakil@ADILs-MacBook-Air MyAssignment % ./patterns

1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 1
Enter the number of rows: 4
Solid Square Pattern (n = 4)
* * * *
* * * *
* * * *
* * * *

1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 2
Enter the number of rows: 3
Enter the number of columns: 5
Solid Rectangular Pattern (m = 3, n = 5)
* * * * *
* * * * *
* * * * *
```

Patterns 3 and 4 (numbers)

```
adilshakil — -zsh — 124x32
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 3
Enter the number of rows: 4
Enter the number of columns: 4
Row-Number Pattern (m = 4, n = 4)
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4

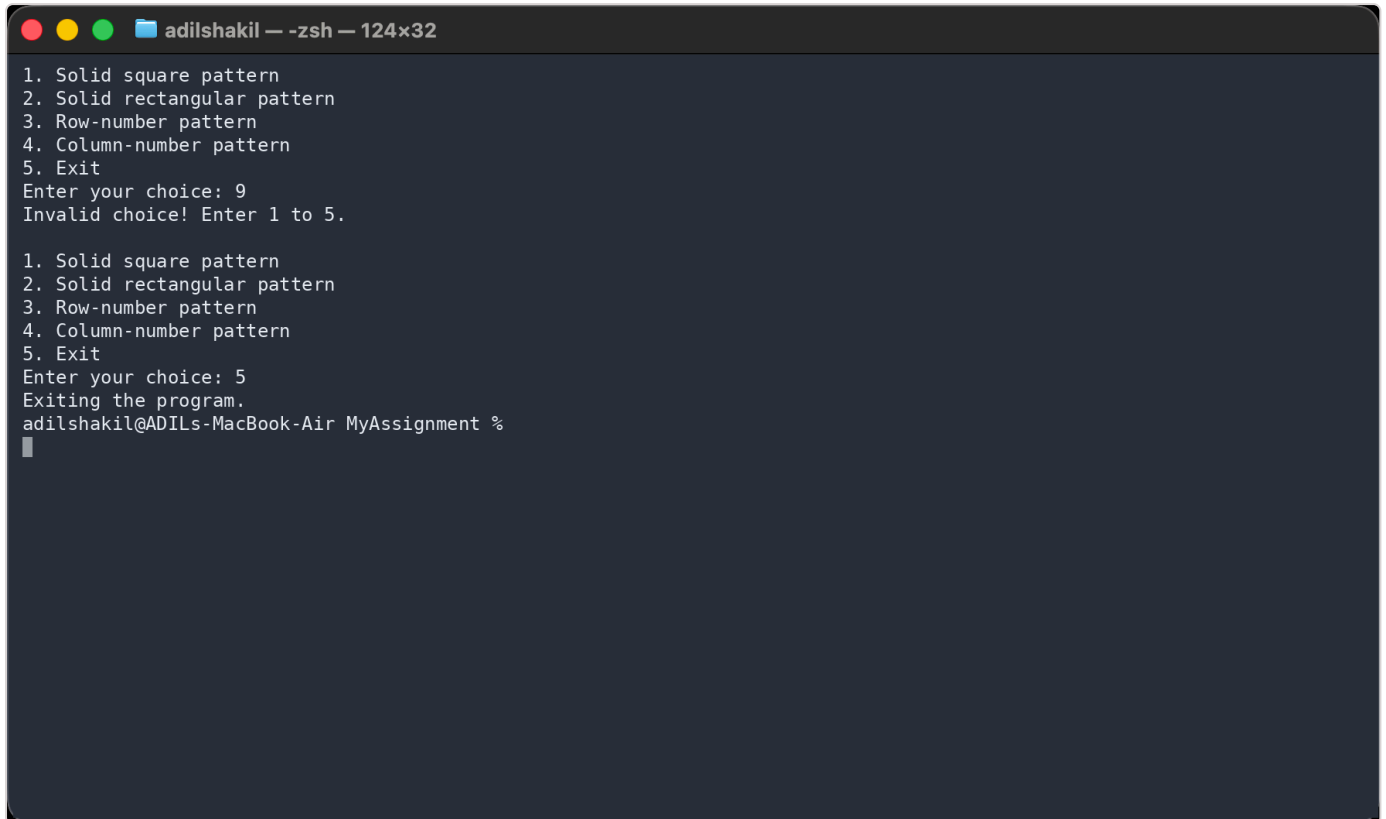
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 4
Enter the number of rows: 4
Enter the number of columns: 4
Column-Number Pattern (m = 4, n = 4)
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4
```

Smallest Inputs and Invalid Inputs

```
adilshakil — -zsh — 124x32
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 1
Enter the number of rows: 1
Solid Square Pattern (n = 1)
*

1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 2
Enter the number of rows: -3
Error: rows must be a positive integer. Enter again: 0
Error: rows must be a positive integer. Enter again: 2
Enter the number of columns: -1
Error: columns must be a positive integer. Enter again: 3
Solid Rectangular Pattern (m = 2, n = 3)
* * *
* * *
```

Invalid Menu Choice and Exit



```
adilshakil — -zsh — 124x32
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 9
Invalid choice! Enter 1 to 5.

1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 5
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment %
```

All twelve test cases gave the expected result. The four patterns match the sample outputs given in the question exactly, and the menu keeps coming back until option 5 is chosen.